Lecture 1: Introduction to AI/ML and Surrogate Modeling for Accelerators

Physics Foundations, Forward Neural Digital Twins, and Closed-Loop Orbit Control

Event: AI Applications in Accelerators Tutorial & Mini-Hackathon
Venue: Korea Multi-purpose Accelerator Complex (KOMAC) / KAERI
Speaker: Dr. Chong Shik Park | **Duration:** 80 Minutes (09:10 – 10:30)
Companion Challenge Notebook: `Single_Notebook_Beam_Orbit_Predictor.ipynb`

Abstract

Modern particle accelerator facilities operate with thousands of tightly coupled magnet actuators and diagnostics under sub-micron beam stability tolerances. Classical linear feedback systems face fundamental bottlenecks in non-linear regimes where magnetic core saturation and power supply limits degrade control fidelity. This lecture provides the theoretical and computational foundation bridging transverse beam dynamics, Hill's equation, and the Orbit Response Matrix (ORM) with cutting-edge supervised machine learning and reinforcement learning. We explore the design and training of neural surrogate digital twins to model cubic saturation, compare classical singular value decomposition (SVD) with model-based bounded optimization (`L-BFGS-B`), and investigate physics-informed closed-loop policy agents with adaptive damping for the afternoon Beam Orbit Alignment Mini-Hackathon.

Contents

1 Lecture Timeline & Curriculum Overview **2**

2 Part 1: Motivation & Accelerator Physics Foundations (15 min) **2**
2.1 The Operational Stability Challenge . . . 2
2.2 Transverse Beam Dynamics & Hill's Equation . . . 3
2.3 Derivation of the Orbit Response Matrix (ORM) . . . 3
2.4 Mini-Hackathon Lattice Geometry & Realistic Non-Linearities . . . 3

3 Part 2: Machine Learning Fundamentals & Neural Surrogate Modeling (25 min) **4**
3.1 Machine Learning Taxonomy in Accelerators . . . 4
3.2 Foundational ML Paradigms: Forward Supervised Learning vs. Reinforcement Learning . . . 4
3.2.1 Forward Supervised Learning: Digital Twin Surrogate & Inversion (Track 1) 4
3.2.2 Reinforcement Learning: Closed-Loop Policy Control (Track 2) . . . 5
3.2.3 Comparative Architectural Analysis . . . 6
3.3 The Artificial Neuron: Mathematical Formulation . . . 6
3.4 Why Non-Linear Activation Functions are Essential . . . 7
3.5 Multi-Layer Perceptron (MLP) Topology: Layers & Information Flow . . . 7
3.6 Loss Functions, Loss Rates & Metrics: MSE vs. RMSE . . . 7
3.7 Optimization Algorithms: Gradient Descent & The Adam Optimizer . . . 8
3.8 Dataset Partitioning & The Bias-Variance Tradeoff . . . 9
3.9 Challenge Implementation & Empirical Benchmark . . . 9

4 Part 3: Closed-Loop Control Paradigms (25 min) **10**
4.1 Baseline 0: Classical SVD Pseudo-Inverse Controller . . . 10
4.2 Baseline 1: Model-Based Optimization via Digital Twin Inversion . . . 10
4.3 Track 2: Closed-Loop RL Policy with Physics Prior and Adaptive Damping . . . 10

5 Part 4: Mini-Hackathon Problem Formulation & Gymnasium Contract (15 min) 11
5.1 Gymnasium Environment Specification (`AcceleratorOrbitGymEnv`) 11
5.2 Standardized 30-Seed Evaluation & Official Scoring Formula 11

6 Part 5: Summary, Q&A & Transition to Lecture 2 (5 min) 11

1 Lecture Timeline & Curriculum Overview

Time	Duration	Core Module Focus
09:10 – 09:25	15 min	**Part 1: Motivation & Accelerator Physics Foundations** Operational stability requirements, Hill's equation, betatron oscillation, ORM derivation, and hackathon lattice geometry.
09:25 – 09:50	25 min	**Part 2: Machine Learning Fundamentals & Neural Surrogate Modeling** Neurons, layers (input/hidden/output), activation functions (ReLU), MSE/RMSE, Adam optimizer, train/val/test splits, bias-variance tradeoff, and MLP digital twins.
09:50 – 10:10	20 min	**Part 3: Closed-Loop Control Paradigms** Baseline 0 (Classical SVD pseudo-inverse), Baseline 1 (SciPy `L-BFGS-B` digital twin inversion), and Track 2 (RL policy agent with dynamic damping).
10:10 – 10:25	15 min	**Part 4: Mini-Hackathon Problem Formulation & Gymnasium Contract** Environment spaces, reward formulation, 30-seed evaluation protocol, scoring formula trade-offs, and live Weights & Biases (W&B) telemetry.
10:25 – 10:30	5 min	**Part 5: Summary, Q&A & Transition to Lecture 2** Key takeaways and transition to Dr. Thorsten Hellert's Agentic AI & Osprey session.

Table 1: Curriculum timeline for the 80-minute morning session.

—

2 Part 1: Motivation & Accelerator Physics Foundations (15 min)

2.1 The Operational Stability Challenge

In synchrotron light sources, free-electron lasers (XFELs), and medical linacs, particle beam stability must be sustained at sub-micron levels to maintain photon brightness and collision luminosity. Orbit distortions arise from multiple sources:

- **Slow Drifts:** Thermal expansion of girders, diurnal ambient temperature shifts, and ground motion.
- **Dynamic Jitter:** Mechanical vibration from cooling water pumps, cryocoolers, and power supply ripple.

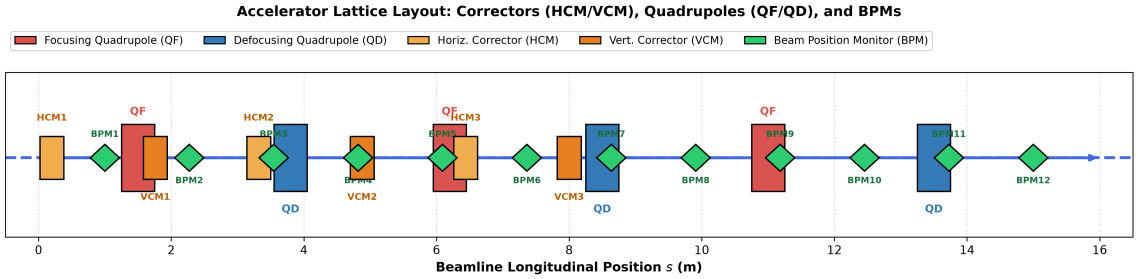


Figure 1: Schematic diagram of the 15-meter accelerator transport section showing alternating focusing (QF, red) and defocusing (QD, blue) quadrupoles, 6 dipole steerers (HCM/VCM), and 12 Beam Position Monitors.

- **High Actuator Coupling:** Deflecting a single dipole corrector causes betatron wave propagation across the entire transport line.

2.2 Transverse Beam Dynamics & Hill's Equation

In an uncoupled linear transport channel, the transverse displacement $x(s)$ as a function of path length s is governed by **Hill's Equation**:

$$\frac{d^2x(s)}{ds^2} + K(s)x(s) = 0 \tag{1}$$

where $K(s) = \frac{e}{p}\frac{\partial B_y(s)}{\partial x}$ represents the alternating quadrupole focusing strength ($K > 0$ for horizontal focusing 'QF', $K < 0$ for defocusing 'QD').

The closed-form betatron solution is given by:

$$x(s) = \sqrt{\epsilon\beta(s)}\cos\left(\psi(s) + \phi_0\right) \tag{2}$$

where $\beta(s)$ is the Courant-Snyder beta function, $\psi(s) = \int_0^s \frac{d\bar{s}}{\beta(\bar{s})}$ is the betatron phase advance, and $\nu = \frac{1}{2\pi}\oint \frac{ds}{\beta(s)}$ is the betatron tune.

2.3 Derivation of the Orbit Response Matrix (ORM)

When an angular kick $\theta_j = \Delta x'_j = \frac{eBL}{p}$ is applied by a dipole corrector at longitudinal position s_j, the transverse displacement registered at a downstream Beam Position Monitor (BPM) at position s_i is linearly proportional to the kick angle:

$$R_{ij} = \frac{\Delta x(s_i)}{\theta_j} = \frac{\sqrt{\beta(s_i)\beta(s_j)}}{2\sin(\pi\nu)}\cos\left(|\psi(s_i) - \psi(s_j)| - \pi\nu\right) \tag{3}$$

For M BPMs and N dipole steerers, the linear response is encapsulated by the matrix $\mathbf{R} \in \mathbb{R}^{N\times M}$:

$$\mathbf{Y}_{\text{linear}} = \mathbf{I}\cdot\mathbf{R} \tag{4}$$

where $\mathbf{I} = [I_1, I_2, \ldots, I_N]$ represents the corrector current vector.

2.4 Mini-Hackathon Lattice Geometry & Realistic Non-Linearities

The mini-hackathon benchmark lattice (Figure 1) models a 15-meter transport line:

- **Actuators:** 6 dipole steerers (3 Horizontal 'HCM1–3', 3 Vertical 'VCM1–3') bounded in $[-5.0\text{ A}, +5.0\text{ A}]$, with single-step adjustments clipped to $[-0.8\text{ A}, +0.8\text{ A}]$.

- **Diagnostics:** 12 Beam Position Monitors ($s = 1.0$ m to 15.0 m) reading beam displacement in millimeters.
- **Magnetic Core Saturation:** Ferromagnetic saturation at higher excitation currents induces non-linear cubic rolloff:

$$\mathbf{Y}_{\text{sat}} = -0.015 \cdot (\mathbf{I}^3 \cdot \mathbf{R}) \tag{5}$$

- **Initial Misalignment Distortion ($\mathbf{Y}_0$):** Quadrupole displacement errors drawn per episode:

$$Y_{0,k} = A_1 \sin\left(\frac{2\pi s_k}{\lambda_1} + \phi_1\right) + A_2 \cos\left(\frac{2\pi s_k}{\lambda_2} + \phi_2\right) \tag{6}$$

  producing initial orbit errors between 0.4 mm and 1.1 mm RMS.
- **Gaussian Measurement Noise:** Sensor electronics introduce random noise $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2)$, with $\sigma = 0.010$ mm (10 μm).

The complete physical forward response equation is:

$$\mathbf{Y} = \mathbf{I} \cdot \mathbf{R} - 0.015 \cdot (\mathbf{I}^3 \cdot \mathbf{R}) + \mathbf{Y}_0 + \boldsymbol{\epsilon} \tag{7}$$

—

3 Part 2: Machine Learning Fundamentals & Neural Surrogate Modeling (25 min)

3.1 Machine Learning Taxonomy in Accelerators

Machine Learning (ML) algorithms automatically extract functional mappings from numerical tracking simulations or machine archives without requiring explicit analytical re-derivation. In accelerator operations, ML encompasses three main paradigms:

- **Supervised Learning:** Learning a continuous regression mapping $\mathbf{Y} = f(\mathbf{X})$ from actuator magnet currents $\mathbf{X}$ to beam diagnostic offsets $\mathbf{Y}$. This forms the foundation of **Surrogate Modeling / Digital Twins**.
- **Unsupervised Learning:** Uncovering latent structures in unlabelled data (e.g., PCA for betatron mode decomposition, autoencoders for anomaly detection).
- **Reinforcement Learning (RL):** Training an agent policy $\pi(a|s)$ through environmental interaction to maximize cumulative reward for **Autonomous Closed-Loop Control**.

3.2 Foundational ML Paradigms: Forward Supervised Learning vs. Reinforcement Learning

In developing autonomous beam alignment algorithms, accelerator engineers rely primarily on two complementary paradigms: **Forward Supervised Learning** (training a digital twin surrogate of the accelerator to be inverted) and **Reinforcement Learning** (training a closed-loop policy agent that directly interacts with the machine). Figure 2 illustrates their distinct architectural workflows.

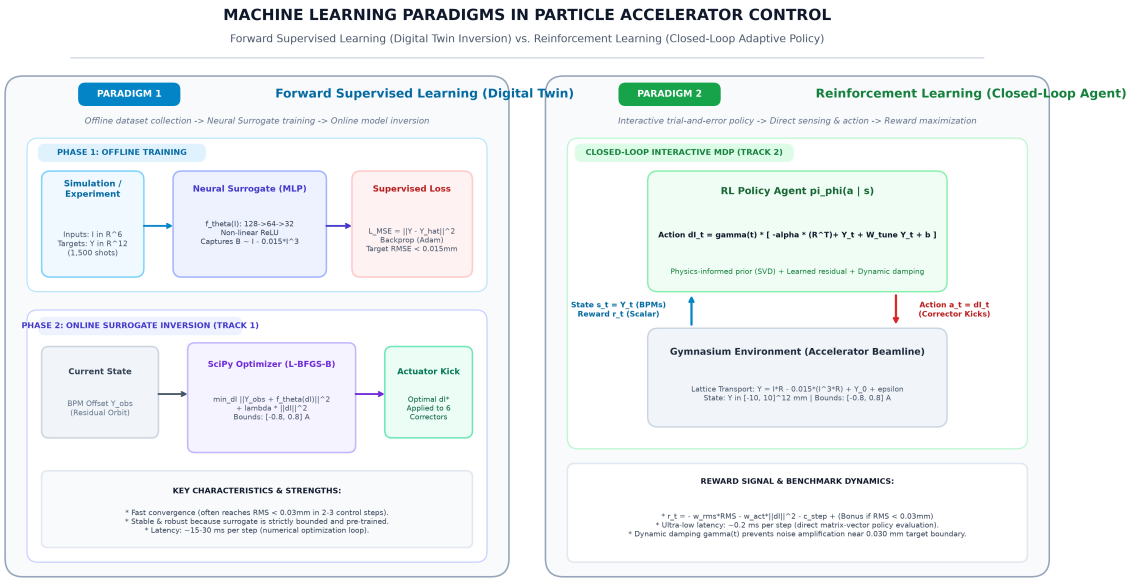


Figure 2: Architectural comparison between Forward Supervised Learning (Surrogate Digital Twin & Inversion, Track 1) and Reinforcement Learning (Closed-Loop Policy Interaction, Track 2) for particle accelerator orbit control.

3.2.1 Forward Supervised Learning: Digital Twin Surrogate & Inversion (Track 1)

In Forward Supervised Learning, the model learns the predictive forward response mapping $\mathbf{I} \mapsto \mathbf{Y}$ from passive, pre-recorded tracking simulation datasets:

1. **Offline Dataset Generation:** We sample N random corrector current vectors $\mathbf{I} \in \mathbb{R}^6$ across operating ranges (e.g., $[-3.0\text{ A}, +3.0\text{ A}]$) and evaluate their transverse displacements $\mathbf{Y} \in \mathbb{R}^{12}$ across all 12 BPMs, yielding dataset $\mathcal{D} = \{(\mathbf{I}^{(i)}, \mathbf{Y}^{(i)})\}_{i=1}^{N}$.

2. **Surrogate Model Training:** A deep neural network (e.g. Multi-Layer Perceptron $f_{\boldsymbol{\theta}}$ with layers $128 \to 64 \to 32$) is trained to minimize the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{Y}^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{I}^{(i)}) \right\|_2^2 \tag{8}$$

   Through Adam backpropagation, the neural surrogate captures non-linear magnetic core saturation $B(I) \approx I - 0.015 \cdot I^3$ down to the 0.010 mm diagnostic noise floor.

3. **Online Inversion for Beam Alignment:** During beam operations, observed orbit distortions $\mathbf{Y}_{\text{obs}}$ are corrected by solving an online bounded minimization problem:

$$\Delta\mathbf{I}^* = \arg\min_{\Delta\mathbf{I}} \|\mathbf{Y}_{\text{obs}} + f_{\boldsymbol{\theta}}(\Delta\mathbf{I})\|_2^2 + \lambda \|\Delta\mathbf{I}\|_2^2 \quad \text{subject to } \Delta\mathbf{I} \in [-0.8\text{ A}, +0.8\text{ A}]^6 \tag{9}$$

   Because the surrogate captures soft saturation, this approach converges in as few as 2 to 3 iterations.

3.2.2 Reinforcement Learning: Closed-Loop Policy Control (Track 2)

In Reinforcement Learning (RL), orbit steering is formulated as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

1. **State Space ($\mathcal{S}$):** Transverse beam offsets measured at 12 BPM stations: $\mathbf{s}_t = \mathbf{Y}_t \in [-10.0 \text{ mm}, +10.0 \text{ mm}]^{12}$.
2. **Action Space ($\mathcal{A}$):** Corrector kick delta currents: $\mathbf{a}_t = \Delta\mathbf{I}_t \in [-0.8 \text{ A}, +0.8 \text{ A}]^6$.
3. **Environment Dynamics ($\mathcal{P}$):** Physical transport law including betatron optics, core saturation, misalignment drift $\mathbf{Y}_0$, and sensor noise $\boldsymbol{\epsilon}$:

$$\mathbf{I}_{t+1} = \text{clip}(\mathbf{I}_t + \mathbf{a}_t, -5.0, +5.0) \text{ A}, \quad \mathbf{s}_{t+1} = \mathbf{I}_{t+1} \cdot \mathbf{R} - 0.015 \cdot (\mathbf{I}_{t+1}^3 \cdot \mathbf{R}) + \mathbf{Y}_0 + \boldsymbol{\epsilon} \tag{10}$$

4. **Reward Signal ($\mathcal{R}$):** Scalar feedback rewarding alignment precision and penalizing actuator effort and steps:

$$r_t = -w_{\text{rms}} \cdot \text{RMS}_{t+1} - w_{\text{act}} \cdot \|\mathbf{a}_t\|_2^2 - c_{\text{step}} + (B_{\text{success}} \text{ if terminated else } 0.0) \tag{11}$$

5. **Physics-Informed Prior & Adaptive Damping:** High-performance policies synthesize an analytical pseudo-inverse prior with learned non-linear residual compensation and dynamic step cooling:

$$\Delta\mathbf{I}_t = \gamma(t) \cdot \left[-\alpha(\mathbf{R}^T)^\dagger \mathbf{Y}_t + \mathbf{W}_{\text{tune}} \mathbf{Y}_t + \mathbf{b} \right], \quad \gamma(t) = \frac{1.0}{1.0 + 0.03 \cdot t} \tag{12}$$

This enables direct policy execution in single forward passes (≈ 0.2 ms latency).

3.2.3 Comparative Architectural Analysis

Table 2 summarizes the fundamental operational differences between the two paradigms.

Dimension	Forward Supervised Learning (Track 1)	Reinforcement Learning (Track 2)
Primary Goal	Accurate physical predictive twin ($\hat{\mathbf{Y}} = f_{\boldsymbol{\theta}}(\mathbf{I})$)	Optimal closed-loop action policy ($\mathbf{a}_t = \pi_{\boldsymbol{\phi}}(\mathbf{s}_t)$)
Data Source	Offline passive sampling across grid	Online interactive rollouts in environment
Training Signal	Ground-truth diagnostic vectors $\mathbf{Y}$	Scalar reward feedback r_t
Objective	Minimize MSE / RMSE error	Maximize cumulative return $\mathbb{E}[\sum \gamma^t r_t]$
Control Mechanism	Forward twin + SciPy `L-BFGS-B` solver	Direct feedback policy evaluation
Execution Latency	$\approx 15 - 30$ ms per step	≈ 0.2 ms per step
Sample Efficiency	High ($1,500$ shots trains full model)	Moderate to high (with physics priors)
Safety Handling	Explicit box constraints in optimizer	Hardware action clipping & damping

Table 2: Comparison of Forward Supervised Learning vs. Closed-Loop Reinforcement Learning for accelerator control.

3.3 The Artificial Neuron: Mathematical Formulation

The fundamental atom of a neural network is the **artificial neuron** (perceptron):

1. **Linear Combination (Pre-Activation):**

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b \tag{13}$$

where $\mathbf{w} \in \mathbb{R}^n$ denotes the trainable synaptic weights, $\mathbf{x} \in \mathbb{R}^n$ is the input feature vector, and $b \in \mathbb{R}$ is the scalar bias shifting the firing threshold.

2. **Non-Linear Mapping (Post-Activation):**

$$a = \sigma(z) = \sigma(\mathbf{w}^T\mathbf{x} + b) \tag{14}$$

where $\sigma(\cdot)$ is the activation function.

3.4 Why Non-Linear Activation Functions are Essential

The Collapse of Deep Linear Networks

If all activation functions were linear identities ($\sigma(z) = z$), stacking L sequential layers would yield:

$$\hat{\mathbf{Y}} = \mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 = (\mathbf{W}_3\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + \mathbf{b}_{\text{net}} = \mathbf{W}_{\text{eff}}\mathbf{x} + \mathbf{b}_{\text{net}} \tag{15}$$

A multi-layer linear network collapses mathematically into a **single linear matrix $\mathbf{W}_{\text{eff}}$**, offering no more expressive capacity than the classical linear Orbit Response Matrix $\mathbf{R}$.

Non-linear activation functions enable neural networks to act as Universal Function Approximators:

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$. Suffers from vanishing gradients when $|z| \gg 0$, halting weight updates during backpropagation.
- **Hyperbolic Tangent (tanh):** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$. Zero-centered, but still prone to gradient saturation.
- **Rectified Linear Unit (ReLU):**

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{16}$$

  Why ReLU is standard in accelerator surrogates:

  - Constant unit derivative $\frac{d}{dz}\text{ReLU}(z) = 1.0$ for $z > 0$, completely preventing gradient vanishing.
  - Extremely fast forward and backward passes without exponential function evaluation.
  - Induces sparse representations by zeroing negative activations.

3.5 Multi-Layer Perceptron (MLP) Topology: Layers & Information Flow

A Multi-Layer Perceptron (MLP) organizes interconnected neurons into sequential layers (Figure 3):

- **Input Layer ($\mathbb{R}^6$):** Ingests physical magnet actuator setpoints (the 6 corrector currents $I_1, \ldots, I_6$).
- **Hidden Layers ($\mathbb{R}^{128} \to \mathbb{R}^{64} \to \mathbb{R}^{32}$):** Progressive hierarchical feature representations. Each layer computes:

$$\mathbf{h}^{(l)} = \text{ReLU}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{17}$$

- **Output Layer ($\mathbb{R}^{12}$):** Linear projection predicting the 12 Beam Position Monitor readings ($\hat{Y}_1, \ldots, \hat{Y}_{12}$).

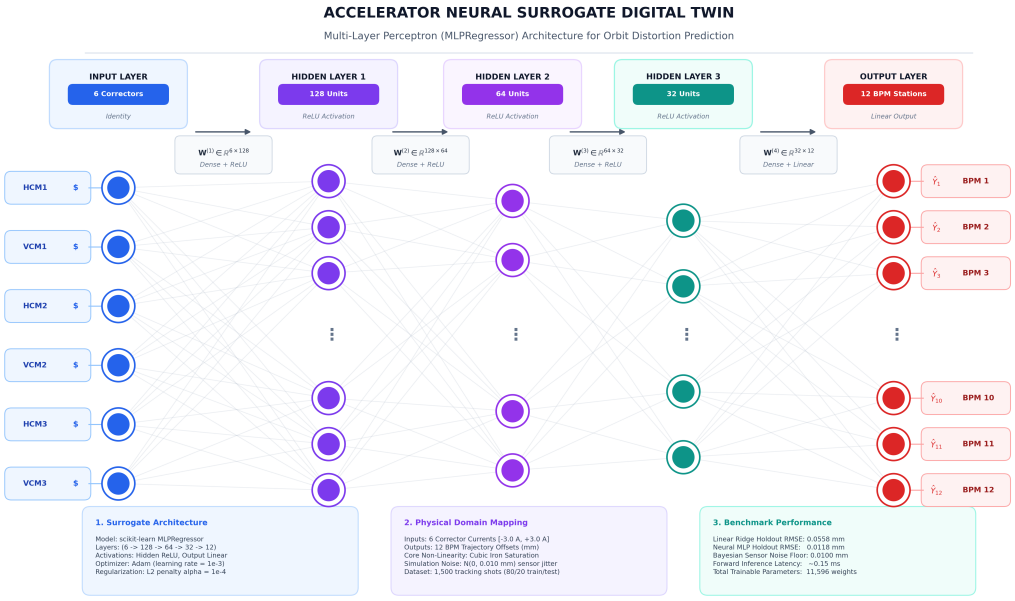


Figure 3: Architecture of the Multi-Layer Perceptron (MLP) neural surrogate digital twin, mapping 6 dipole corrector currents to 12 BPM transverse orbit offsets through three hidden layers (128, 64, 32 units) with ReLU activations.

3.6 Loss Functions, Loss Rates & Metrics: MSE vs. RMSE

Network parameters are trained by minimizing an objective loss function:

1. **Mean Squared Error (MSE) Loss:**

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) = \frac{1}{N \cdot M} \sum_{i=1}^{N} \sum_{j=1}^{M} \left(Y_{i,j} - \hat{Y}_{i,j}(\boldsymbol{\theta}) \right)^2 \tag{18}$$

   Smoothly differentiable, penalizing large orbit outliers quadratically.

2. **Root Mean Squared Error (RMSE) Evaluation Metric:**

$$\text{RMSE} = \sqrt{\mathcal{L}_{\text{MSE}}} = \sqrt{\frac{1}{N \cdot M} \sum_{i=1}^{N} \sum_{j=1}^{M} \left(Y_{i,j} - \hat{Y}_{i,j} \right)^2} \tag{19}$$

   Why RMSE is the preferred metric in beam physics: It preserves the original physical measurement units (**millimeters**) and directly reflects accelerator beam orbit distortion standards.

3.7 Optimization Algorithms: Gradient Descent & The Adam Optimizer

During backpropagation, loss gradients $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ are propagated backward to update weights:

1. **Stochastic Gradient Descent (SGD):** $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$, where η is the **learning rate**. High learning rates risk divergence; low learning rates cause stagnation.

2. **The Adam Optimizer (Adaptive Moment Estimation):** Combines Momentum (first moment $\mathbf{m}_t$) and RMSprop (second moment $\mathbf{v}_t$):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \tag{20}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \implies \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}\hat{\mathbf{m}}_t \tag{21}$$

Adam automatically scales individual parameter updates, ensuring stable convergence across coupled magnet channels.

3.8 Dataset Partitioning & The Bias-Variance Tradeoff

To prevent memorization of diagnostic noise, data is partitioned into:

- **Training Set (80%, 1,200 shots):** Used directly for loss gradient computation and parameter updates.
- **Validation Set (Optional):** Monitored across epochs for early stopping before overfitting occurs.
- **Test / Holdout Set (20%, 300 shots):** Strictly isolated during training to provide an unbiased benchmark on unseen accelerator states.
- **Underfitting (High Bias):** Linear models cannot capture the cubic magnetic core saturation.
- **Overfitting (High Variance):** Over-parameterized models memorizing Gaussian sensor noise ($\sigma = 10$ μm). Prevented using L_2 weight regularization (‘alpha=1e-4‘).

3.9 Challenge Implementation & Empirical Benchmark

In Step 3 of the challenge notebook, $1,500$ experimental samples are synthesized uniformly across current boundaries $[-3.0\text{ A}, +3.0\text{ A}]$:

```
n_train_samples = 1500
X_offline = np.random.uniform(-3.0, 3.0, size=(n_train_samples, env.
    n_correctors))
Y_linear = X_offline @ env.R
Y_sat = - 0.015 * (X_offline**3) @ env.R
bpm_noise = np.random.normal(0, 0.010, size=(n_train_samples, env.n_bpms))
Y_offline = Y_linear + Y_sat + bpm_noise

X_train, X_test, Y_train, Y_test = train_test_split(
    X_offline, Y_offline, test_size=0.20, random_state=42
)

# Train Linear Ridge vs Deep MLP Regressor
linear_model = Ridge(alpha=1.0).fit(X_train, Y_train)
surrogate_model = MLPRegressor(
    hidden_layer_sizes=(128, 64, 32), activation='relu',
    solver='adam', alpha=1e-4, max_iter=400, random_state=42
).fit(X_train, Y_train)
```

Model Architecture	Parameters	Holdout RMSE	Inference Latency	Physical Regimes Captured
Linear Ridge Baseline ($\lambda = 1.0$)	72	0.0558 mm	< 0.01 ms	Linear optics only; misses saturation.
MLP Neural Surrogate	11,596	**0.0118** mm	≈ 0.15 ms	Linear ORM and cubic saturation down to noise floor.

Table 3: Holdout validation performance on 20% test partition (300 unseen shots).

Physical Insight: The Bayesian Sensor Noise Floor

The holdout error of the MLP (0.0118 mm) closely approaches the theoretical noise floor set by Gaussian BPM sensor jitter ($\sigma = 0.0100$ mm). This indicates that the neural surrogate has learned the non-linear physics without overfitting sensor noise.

—

4 Part 3: Closed-Loop Control Paradigms (25 min)

4.1 Baseline 0: Classical SVD Pseudo-Inverse Controller

In standard accelerator control rooms, Singular Value Decomposition (SVD) of the transpose response matrix $\mathbf{R}^T = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ provides the Moore-Penrose pseudo-inverse:

$$(\mathbf{R}^T)^\dagger = \mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^T \tag{22}$$

The closed-loop correction action per step is:

$$\Delta\mathbf{I} = \text{clip}\left(-\alpha \cdot (\mathbf{R}^T)^\dagger \cdot \mathbf{Y}_{\text{obs}},\ -0.8,\ +0.8\right) \tag{23}$$

where $\alpha \approx 0.45$ is the feedback gain. While computationally instantaneous, SVD assumes linearity; when correctors saturate, high gains trigger overshooting and limit-cycle oscillations.

4.2 Baseline 1: Model-Based Optimization via Digital Twin Inversion

Rather than inverting an analytical matrix, Baseline 1 optimizes corrective kicks directly through the forward neural surrogate $f_{\boldsymbol{\theta}}(\Delta\mathbf{I})$ using SciPy's `L-BFGS-B` algorithm:

$$\min_{\Delta\mathbf{I}} \mathcal{L}(\Delta\mathbf{I}) = \|\mathbf{Y}_{\text{obs}} + f_{\boldsymbol{\theta}}(\Delta\mathbf{I})\|_2^2 + \lambda\|\Delta\mathbf{I}\|_2^2 \quad \text{s.t.} \quad \Delta\mathbf{I} \in [-5.0\text{ A}, +5.0\text{ A}]^6 \tag{24}$$

```python
class SupervisedMLPController:
    def __init__(self, surrogate):
        self.surrogate = surrogate

    def act(self, obs, info=None):
        def objective(delta_I):
            pred = self.surrogate.predict(delta_I.reshape(1, -1))[0]
            residual = obs + pred
            return np.sum(residual**2) + 0.0001 * np.sum(delta_I**2)

        res = opt.minimize(objective, np.zeros(6), bounds=[(-5.0, 5.0)]*6,
    method='L-BFGS-B')
        return np.clip(res.x, -0.8, 0.8)
```

Because the surrogate anticipates non-linear saturation falloff, Baseline 1 reaches convergence in only **2 to 3 iterations**.

4.3 Track 2: Closed-Loop RL Policy with Physics Prior and Adaptive Damping

Model-based iterative optimization requires 15–30 ms per step. A closed-loop policy network executes in < 0.5 ms. High-performing accelerator policy agents integrate an analytical **physics prior** with a learned parameter residual and dynamic step damping:

$$\Delta\mathbf{I}(t) = \gamma(t) \cdot \left[\underbrace{-\alpha(\mathbf{R}^T)^\dagger\mathbf{Y}}_{\text{Analytical Physics Prior}} + \underbrace{\mathbf{W}_{\text{learned}}\mathbf{Y} + \mathbf{b}}_{\text{Learned Residual Offset}} \right] \tag{25}$$

$$\gamma(t) = \frac{1.0}{1.0 + 0.03 \cdot t} \tag{26}$$

The dynamic damping factor $\gamma(t)$ decays control authority as the beam approaches the 0.030 mm boundary, preventing sensor noise amplification from destabilizing the closed orbit.

—

5 Part 4: Mini-Hackathon Problem Formulation & Gymnasium Contract (15 min)

5.1 Gymnasium Environment Specification (`AcceleratorOrbitGymEnv`)

- **Observation Space:** $\text{Box}(-10.0, +10.0, \text{shape} = (12,), \text{dtype} = \text{float32})$ (BPM readings in mm).
- **Action Space:** $\text{Box}(-0.8, +0.8, \text{shape} = (6,), \text{dtype} = \text{float32})$ (Incremental kicks $\Delta\mathbf{I}$ in Amperes).
- **State Update:** $\mathbf{I}_{t+1} = \text{clip}(\mathbf{I}_t + \Delta\mathbf{I}_t, -5.0, 5.0)$.
- **Reward Function:**

$$r_t = -\text{RMS}_{t+1} - 0.01 \cdot \|\Delta\mathbf{I}_t\|_2 + (50.0 \text{ if } \text{RMS}_{t+1} < 0.030 \text{ mm else } 0.0) \tag{27}$$

- **Success Criteria:** Terminated when RMS < 0.030 mm; truncated if step count reaches 20.

5.2 Standardized 30-Seed Evaluation & Official Scoring Formula

Controllers are evaluated across 30 standardized seeds (`range(1, 31)`). The official benchmark score is defined as:

$$\text{Score} = (\text{Success Rate} \times 10.0) - (2.0 \times \overline{\text{Steps}}) + \left(\frac{0.03}{\max(\overline{\text{RMS}}, 0.005)} \times 20.0\right) - (2.0 \times \overline{\|\mathbf{I}\|}) \tag{28}$$

Rank	Controller Paradigm	Success Rate	Mean Steps	Final RMS	Current Norm	Official Score
#1	Baseline 1: Supervised MLP Inversion	100.0%	**2.3 steps**	0.0257 mm	1.21 A	**1,016.8 pts**
#2	Baseline 0: Classical Linear SVD	100.0%	5.6 steps	**0.0230 mm**	**1.16 A**	**1,013.1 pts**
#3	Track 2: Closed-Loop RL Policy	100.0%	6.1 steps	0.0239 mm	1.18 A	**1,011.1 pts**

Table 4: Official benchmark results across 30 evaluation seeds.

—

6 Part 5: Summary, Q&A & Transition to Lecture 2 (5 min)

Summary Takeaways

1. **Physics-Informed Prior:** Initializing AI policies with the analytical pseudo-inverse ORM guarantees early stability and avoids millions of exploratory random steps.
2. **Non-Linear Representation:** Deep neural surrogates bridge the gap between idealized linear beam optics and real-world magnetic saturation.
3. **Dynamic Step Damping:** Attenuating feedback near diagnostic noise floors

prevents terminal limit cycles and ensures robust convergence.

Next Session: Dr. Thorsten Hellert will present *Agentic AI for Accelerator Research: Osprey Applications* (10:40 – 12:00), exploring LLM agents, automated tool calling, and autonomous accelerator orchestration.

References:

1. Minty, M. G., & Zimmermann, F. (2003). *Measurement and Control of Charged Particle Beams.* Springer.
2. Wiedemann, H. (2015). *Particle Accelerator Physics* (4th ed.). Springer.
3. Hellert, T., et al. (2024). *Autonomous Accelerator Tuning using Large Language Model Agents and Osprey.* Phys. Rev. Accel. Beams.
4. Edelen, A., et al. (2020). *Machine learning for orders-of-magnitude speedup in multiobjective optimization of particle accelerator beamlines.* Phys. Rev. Accel. Beams, 23(4), 044601.